FICE SELECTION COMMITTEE

Технічний звіт

v1.2.0

Дата: 30 березня 2026

Підготовлено: Release Engineering Team

Попередня версія: v1.1.0

Зміст

1. Загальний огляд

2. Що було до змін (v1.1.0)

3. Що саме змінено

- 3.1 Docker: Перехід на багатоетапні збірки
- 3.2 Авторизація: POST → GET для getCurrentUser
- 3.3 Інфраструктура: Docker Compose
- 3.4 CI/CD: Розширена матриця та сканування вразливостей
- 3.5 Telegram Bot: Новий сервіс
- 3.6 Моніторинг: Grafana
- 3.7 Документація

4. Нові бібліотеки та модулі

5. Як тепер працює система

6. Як запускати

7. Як деплоїти

8. Як моніторити

9. Як тестувати

10. Порівняння «Було / Стало»

11. Висновки та подальші кроки

1. Загальний огляд

> **Реліз v1.2.0** — це масштабне оновлення інфраструктури та операційної зрілості платформи FICE Selection Committee. Основний фокус: надійність збірок, коректність контрактів, розширення функціональності та спостережуваність системи.

Цей реліз приносить ключові покращення в кількох напрямках:

- **Багатоетапні Docker-збірки** — триетапний процес побудови образів, що забезпечує відтворюваність та мінімальний розмір runtime-образу.
- **Виправлення Auth API** — зміна HTTP-методу `getCurrentUser` з `POST` на `GET` для семантичної коректності.
- **Новий Telegram Bot сервіс** — управління feature-прапорцями через Telegram з підтримкою webhook, inline-клавіатур та i18n.
- **Розширений CI/CD** — нова матриця збірок, сканування вразливостей Trivy, семантичне тегування Docker-образів.
- **Моніторинг Grafana** — 4 попередньо налаштовані дашборди для операційного моніторингу.
- **Інженерна документація** — CLAUDE.md та 10 довідкових файлів як стандарт інженерної роботи.

7+1	79	4	3
мікросервісів + шлюз	файлів у Telegram Bot	Grafana дашборди	етапи Docker-збірки

2. Що було до змін (v1.1.0)

Стан системи до релізу v1.2.0 характеризувався наступними обмеженнями та особливостями:

Аспект	Стан у v1.1.0
Мікросервіси	6 сервісів (identity, admission, documents, environment, notifications, gateway) + Next.js фронтенд
Dockerfile	Одноетапні збірки на базі `gradle:jdk21-ubi-minimal`, Gradle-скрипти копіювались у директорію кожного сервісу
Docker Compose	`service_started` для некритичних залежностей (Redis, RabbitMQ) — без перевірки здоров'я
CI/CD	Тригери лише на `server/services/**`, без сканування вразливостей
Auth API	`POST /api/v1/auth/user` — семантично некоректний метод для читання даних
Telegram	Інтеграція відсутня
Моніторинг	Без провізійованих Grafana дашбордів
Онбордінг розробника	Без `.env.example` шаблону
Інженерна документація	Відсутня — немає стандартів та довідкових файлів
Планування задач	14 загальних фаз (phase-01-scaffolding – phase-14-polish)

3. Що саме змінено

3.1 Docker: Перехід на багатоетапні збірки ЗМІНЕНО

Усі сервіси переведено на триетапну Docker-збірку, що забезпечує ізоляцію етапів, мінімальний runtime-образ та правильний порядок побудови спільних бібліотек.

1

lib-builder

`eclipse-temurin:21-jdk-alpine`

Побудова спільних бібліотек (`publishToMavenLocal`)

2

service-builder

Збірка JAR сервісу з використанням бібліотек з етапу 1

3

runtime

`eclipse-temurin:21-jre-alpine`

Мінімальний образ + `HEALTHCHECK`

Ключові рішення:

- Базовий образ змінено з `gradle:jdk21-ubi-minimal` на `eclipse-temurin:21-jdk-alpine` (збірка) та `eclipse-temurin:21-jre-alpine` (runtime)
- Стандартизований користувач `spring` з UID/GID `10000/10001` у всіх сервісах
- Додано `server/.dockerignore` для оптимізації контексту збірки
- Вбудований `HEALTHCHECK` для кожного контейнера

Приклад структури Dockerfile:

```dockerfile
# Етап 1: Побудова спільних бібліотек
FROM eclipse-temurin:21-jdk-alpine AS lib-builder
WORKDIR /workspace
COPY libs/ libs/
COPY buildSrc/ buildSrc/
RUN ./gradlew :libs:sc-auth-starter:publishToMavenLocal

# Етап 2: Збірка сервісу
FROM eclipse-temurin:21-jdk-alpine AS service-builder
COPY --from=lib-builder /root/.m2 /root/.m2
RUN ./gradlew :services:my-service:bootJar

# Етап 3: Runtime
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -g 10001 spring && adduser -u 10000 -G spring spring
COPY --from=service-builder /workspace/build/libs/*.jar app.jar
HEALTHCHECK CMD wget -qO- http://localhost:8080/actuator/health || exit 1
```

3.2 Авторизація: POST → GET для getCurrentUser ЗМІНЕНО

Ендпоінт отримання поточного користувача змінено з `POST` на `GET` для дотримання REST-семантики — операція читання даних не повинна використовувати метод, що передбачає створення ресурсу.

Компонент	Зміна
`IdentityServiceClient`	`@PostMapping` → `@GetMapping` для `/api/v1/auth/user`
`ProviderContractIT`	Оновлено: тепер використовує потік magic link автентифікації
`IdentityClientContractIT` (documents)	Контрактний тест оновлено відповідно до нового методу

> **Зверніть увагу:** Це ламаюча зміна (breaking change) для будь-яких зовнішніх клієнтів, що використовують `POST /api/v1/auth/user`. Усі внутрішні Feign-клієнти та

> контрактні тести оновлено.

3.3 Інфраструктура: Docker Compose ПОКРАЩЕНО

Значні зміни у конфігурації Docker Compose для підвищення надійності та підтримки нових сервісів:

- **Залежності на основі здоров'я:** `service_started` → `service_healthy` для Redis та RabbitMQ — сервіси чекають на фактичну готовність інфраструктури
- **Контекст збірки:** змінено на кореневу директорію `server/` для підтримки багатоетапних збірок з доступом до спільних бібліотек
- **JWT секрети:** volume mounts для identity-service
- **Rate limiting:** конфігурація обмеження частоти для magic link
- **Telegram Bot:** новий сервіс `telegram-bot-service` + `ngrok` + `webhook-init` у профілі `telegram`
- **Іменування змінних:** виправлено `RABBIT_*` → `RABBITMQ_*`
- **Health check шлюзу:** виправлено порт з `8080` → `8079` (management port)
- **.env.example:** створено шаблон з документацією усіх змінних оточення

3.4 CI/CD: Розширена матриця та сканування вразливостей

ПОКРАЩЕНО

Workflow `service-docker.yml` суттєво розширено:

Можливість	Деталі
Тригери	Додано: зміни у `libs/`, `buildSrc/`, конфігах збірки; теги `v*.*.*`
Матриця збірки	Додано: `web` (фронтенд) та `telegram-bot`
Сканування вразливостей	Trivy з вивантаженням SARIF-звітів до GitHub Security
Контекст/Dockerfile	Динамічне визначення контексту та Dockerfile для кожного типу сервісу
Таймаут	30-хвилинний ліміт виконання job
Тегування образів	Семантичне версіонування (semver) Docker-тегів

3.5 Telegram Bot: Новий сервіс НОВИЙ

Повністю новий Spring Boot мікросервіс для оперативного управління feature-прапорцями через Telegram:

> **79 файлів** вихідного коду з повним покриттям юніт-тестами. Підтримка української та англійської мов (i18n).

Функціональність	Опис
Webhook інтеграція	Inline-клавіатури для інтерактивної взаємодії з ботом
Feature Flags CRUD	Створення, читання, оновлення та видалення прапорців через Feign-клієнт до environment-service
RabbitMQ підписки	Сповіщення про зміни feature-прапорців через чергу повідомлень
Авторизація	Авторизація на основі chat ID
Rate Limiting	Обмеження частоти запитів для захисту від зловживань
Здоров'я	Health indicator для перевірки доступності Telegram API
Міграції БД	3 Flyway-міграції для ініціалізації схеми
i18n	Локалізація: українська (uk) та англійська (en)

3.6 Моніторинг: Grafana НОВИЙ

Додано автоматичне провізіонування Grafana з попередньо налаштованими дашбордами:

Дашборд	Призначення
Services Overview	Загальний стан усіх мікросервісів, HTTP-метрики, JVM
PostgreSQL	Метрики баз даних: з'єднання, транзакції, розмір
RabbitMQ	Черги, повідомлення, швидкість обробки, споживачі
Feature Flags	Статус прапорців, історія змін, використання

Конфігурація включає провізіонування джерел даних (Prometheus) та автоматичне виявлення дашбордів.

3.7 Документація НОВИЙ

- **CLAUDE.md** — інженерний операційний стандарт: агенти, навички, правила, формат пропозицій змін
- **docs/claude/** — 10 довідкових файлів:
  - `architecture.md` — топологія сервісів, бібліотеки, інфраструктура
  - `testing.md` — TDD-протокол, тестова інфраструктура
  - `docker.md` — Docker-робочі процеси, відомі проблеми
  - `build.md` — команди збірки, ланцюжок залежностей
  - `agents.md`, `skills.md` — специфікації агентів та навичок
  - `interaction-standards.md`, `ios-atmosphere.md` — стандарти UI/UX
  - та інші
- **REVIEW_REPORT.md** — аудит з 48 знахідками (UI/UX, доступність, продуктивність, безпека)
- **Оновлені плани задач:** 19 нових файлів замість 14 старих, включаючи специфікації редизайну

4. Нові бібліотеки та модулі

Модуль / Бібліотека	Тип	Опис
`selection-committee-telegram-bot-service`	СЕРВІС	Spring Boot мікросервіс для Telegram-інтеграції з управлінням feature-прапорцями
Gradle buildSrc conventions	ЗБІРКА	`sc.java-conventions`, `sc.spring-boot-service`, `sc.testing-conventions`, `sc.jacoco-conventions`
Root `package.json`	ІНСТРУМЕНТ	shadcn CLI для управління UI-компонентами фронтенду
Trivy Scanner	БЕЗПЕКА	Сканування вразливостей Docker-образів у CI з вивантаженням SARIF-звітів
ngrok	ІНТЕГРАЦІЯ	Тунелювання для Telegram webhook у локальному середовищі розробки

5. Як тепер працює система

Після релізу v1.2.0 система складається з 7 мікросервісів, API-шлюзу, веб-фронтенду та повного стеку інфраструктури:

Мікросервіси

Identity Service Автентифікація, magic link, JWT	**Admission Service** Заяви абітурієнтів, конкурс	**Documents Service** Зберігання документів, MinIO
Environment Service Feature flags, конфігурація	**Notifications Service** Email-сповіщення, Mailpit	**Telegram Bot Service** Telegram-бот, feature flags
Gateway API-шлюз, порт 8080	**Web (Next.js)** Фронтенд, SPA	

Інфраструктура

PostgreSQL 16 Основна СУБД	**Redis 7** Кешування, сесії	**RabbitMQ 3** Брокер повідомлень
MinIO Об'єктне сховище	**Mailpit** Тестовий SMTP	**Zipkin** Розподілене трасування
Prometheus Збір метрик	**Grafana** Візуалізація, дашборди	

Архітектурні принципи v1.2.0

- Усі сервіси використовують **багатоетапні Docker-збірки** з попередньою публікацією спільних бібліотек

- **Координація залежностей на основі здоров'я** (`service_healthy`) для всіх сервісів
- **Magic link автентифікація** — без паролів, лише email-посилання
- Feature flags керуються через **environment-service** та доступні через **Telegram-бот**
- **Grafana дашборди** для операційного моніторингу провізіонуються автоматично
- Кожен сервіс має **HEALTHCHECK** у Dockerfile та `/actuator/health` ендпоінт

6. Як запускати

Підготовка середовища

```
# Скопіюйте шаблон змінних оточення та заповніть значення
cp infra/.env.example infra/.env
```

Запуск інфраструктури

```
# Запуск PostgreSQL, Redis, RabbitMQ, MinIO, Mailpit, Zipkin, Prometheus, Gr
afana
cd infra && docker compose up -d
```

Запуск сервісів

```
# Запуск усіх основних мікросервісів
docker compose -f docker-compose.services.yml up -d
```

Запуск з Telegram Bot (опціонально)

```
# Включає telegram-bot-service + ngrok + webhook-init
docker compose -f docker-compose.services.yml --profile telegram up -d
```

> **Передумови:** Docker та Docker Compose мають бути встановлені. Усі сервіси запускаються у мережі `sc-net`. Перша збірка може тривати 5-10 хвилин через завантаження залежностей.

7. Як деплоїти

Тригер	Дія
Push до `main`	Запускає Docker CI — збірка та публікація образів із змінених сервісів
Тег `v*.*.*`	Повна перезбірка усіх сервісів з тегуванням за semver
Публікація образів	`ghcr.io/fice-selection-committee/sc-*`
Сканування Trivy	Виконується на кожній збірці, результати у GitHub Security (SARIF)

Процес деплою:

1. Зміни проходять code review та CI-перевірки у pull request
2. Після merge до `main` автоматично збираються Docker-образи змінених сервісів
3. Trivy сканує кожен образ на вразливості та завантажує SARIF-звіт
4. Для релізу створюється тег `v*.*.*`, що тригерить повну перезбірку
5. Семантичні теги (`latest` , `v1.2.0` , `v1.2` , `v1`) застосовуються до образів

8. Як моніторити

Інструмент	URL	Призначення
Grafana	`http://localhost:3001`	Візуалізація метрик (логін: `admin/admin`)
Zipkin	`http://localhost:9411`	Розподілене трасування запитів між сервісами
Prometheus	`http://localhost:9090`	Збір та запити метрик
Actuator	`/actuator/health`	Перевірка здоров'я кожного сервісу

Grafana дашборди

1. **Services Overview** — HTTP-метрики, час відповіді, JVM-статистика усіх сервісів
2. **PostgreSQL** — кількість з'єднань, транзакцій, розмір баз даних
3. **RabbitMQ** — стан черг, швидкість обробки повідомлень, кількість споживачів
4. **Feature Flags** — поточний статус прапорців, історія змін

> Усі дашборди провізіонуються автоматично при першому запуску Grafana. Джерело даних Prometheus налаштовується через `infra/grafana/provisioning/`.

9. Як тестувати

Рівень тестування	Команда	Вимоги
Unit-тести	`./gradlew test`	Запускаються для кожного сервісу окремо
Integration-тести	`./gradlew integrationTest`	Потребують Docker (Testcontainers)
Contract-тести	Частина `integrationTest`	У `src/integrationTest/**/contract/`
E2E-тести	Playwright	У `selection-committee-e2e-tests`

Telegram Bot тести

Сервіс Telegram Bot включає 79 файлів з повним покриттям юніт-тестами. Тести перевіряють:

- Обробку webhook-запитів та inline-клавіатур
- CRUD-операції з feature-прапорцями через Feign-клієнт
- Обробку RabbitMQ-повідомлень
- Авторизацію на основі chat ID
- Rate limiting
- i18n (українська та англійська локалізації)

Контрактні тести (оновлені)

- `ProviderContractIT` — оновлено для magic link потоку автентифікації
- `IdentityClientContractIT` (documents-service) — оновлено для `GET /api/v1/auth/user`
- `EnvClientContractIT` — видалено (застарілий)

10. Порівняння «Було / Стало»

Аспект	v1.1.0 (Було)	v1.2.0 (Стало)
Docker-збірки	~~Одноетапні (gradle:jdk21-ubi-minimal)~~	Багатоетапні, 3 стадії (eclipse-temurin:21-alpine)
Залежності здоров'я	~~service_started~~	service_healthy
Auth API	~~POST /api/v1/auth/user~~	GET /api/v1/auth/user
Кількість сервісів	~~6 + gateway~~	7 + gateway
CI тригери	~~server/services/** тільки~~	libs, buildSrc, теги, web, telegram-bot
Сканування вразливостей	~~Відсутнє~~	Trivy SARIF до GitHub Security
Дашборди моніторингу	~~Ручне налаштування~~	4 провізійовані Grafana дашборди
Онбордінг розробника	~~Без .env шаблону~~	.env.example з документацією
Інженерна документація	~~Відсутня~~	CLAUDE.md + 10 довідкових файлів
Планування задач	~~14 загальних фаз~~	19 специфічних планів + специфікації редизайну

11. Висновки та подальші кроки

Ключові досягнення v1.2.0

- **Багатоетапні Docker-збірки** зменшують розмір образів та покращують відтворюваність збірок. Виключення JDK та Gradle з runtime-образу значно зменшує поверхню атаки.
- **Залежності на основі здоров'я** запобігають стартовим гонкам (race conditions) — сервіси не починають роботу, поки інфраструктура не підтвердить свою готовність.
- **Виправлення Auth API** приводить контракт у відповідність до REST-семантики, що спрощує інтеграцію для зовнішніх клієнтів.
- **Telegram Bot** додає зручний операційний інтерфейс для управління feature-прапорцями без необхідності доступу до адмін-панелі.
- **Grafana дашборди** забезпечують миттєву спостережуваність системи з першого запуску.
- **Trivy у CI** встановлює безперервний контроль вразливостей як частину процесу збірки.
- **Інженерна документація** формалізує стандарти роботи, що підвищує передбачуваність та якість змін.

Подальші кроки

Пріоритет	Задача	Опис
ВИСОКИЙ	PR Review та Merge	Ревью та злиття поточної гілки з усіма змінами v1.2.0
ВИСОКИЙ	Тегування v1.2.0	Створення git-тегу `v1.2.0` після merge для публікації Docker-образів
СЕРЕДНІЙ	Редизайн фронтенду	Виконання планів redesign-01 – redesign-07 для оновлення UI
СЕРЕДНІЙ	Виправлення доступності	Реалізація знахідок з REVIEW_REPORT.md щодо accessibility
ПЛАНОВИЙ	E2E покриття Telegram Bot	Інтеграційні тести з реальним Telegram API (тестове оточення)
ПЛАНОВИЙ	Кешування Docker-шарів	Оптимізація CI через кешування між етапами збірки

Реліз v1.2.0 значно підвищує операційну зрілість платформи FICE Selection Committee. Інфраструктурні покращення створюють надійний фундамент для подальшого розвитку функціональності та масштабування системи.